IT APPLICATION-BASED PROBLEM SOLVING

SCALABILITY AUDIT OF AN E-COMMERCE PLATFORM

Course: Design and Analysis of Algorithms (DAA)
Assessment: CO1 – IT Application-Based Problem Solving
Case Study: Scalability Audit of ShopSphere E-Commerce Platform

Submitted by:
Name: _POLNATI ABHIRAMA KRISHNA
Register Number: 192472037
Department: CSE – Artificial Intelligence
Year / Section: 3rd year

Submitted to:
Faculty Name: Dr.Anoop

Institution: SIMATS ENGINEERING

Submission Date:13/10/2026_

ABSTRACT

The growth of an e-commerce platform introduces significant computational challenges in product management, API request processing, product searching, recommendation generation, and sales analytics. Algorithms that perform efficiently on small datasets may become impractical when the number of products, requests, or transactions increases significantly.

This report presents a scalability audit of a hypothetical e-commerce platform named **ShopSphere**. Five computational modules are examined: duplicate product detection, API rate limiting, product searching, recommendation generation, and sales analytics.

The analysis applies algorithmic complexity, recurrence relations, the Master Theorem, recurrence-tree analysis, and asymptotic notation. Exact operation counts are calculated wherever possible, while recurrence-based estimates are used for recursive algorithms. The algorithms are also evaluated using an assumed server processing rate of 10^8basic operations per second and a nightly processing window of 600 seconds.

The results identify computational bottlenecks and demonstrate how suitable algorithmic redesigns can improve performance. The report concludes with recommendations for building scalable and efficient e-commerce systems.

Keywords: Design and Analysis of Algorithms, Time Complexity, Space Complexity, Big-O Notation, Master Theorem, Recurrence Relations, Scalability, E-Commerce.

1. INTRODUCTION

1.1 Background

An e-commerce platform manages a large amount of data, including product information, customer requests, product recommendations, orders, payments, and sales records.

As the platform grows, the number of operations performed by its algorithms also increases. An algorithm that works well for 100 products may become too slow when processing one million products.

For example, comparing every product with every other product requires a large number of comparisons. Similarly, a poorly designed recommendation algorithm may perform excessive recursive computations.

Therefore, analysing algorithmic efficiency is necessary to ensure that an application remains responsive and reliable as its data grows.

1.2 Problem Statement

ShopSphere is experiencing performance challenges because its computational modules must process increasingly large datasets.

The main problems are:

1. Duplicate product detection may perform an excessive number of comparisons.
2. API rate-limiting logic must audit requests efficiently.
3. Product searching must remain fast when the catalogue contains millions of products.
4. Recommendation generation must avoid unnecessary recursive computations.
5. Sales analytics must process large datasets within a limited execution window.

The objective of this assessment is to analyse these modules mathematically, estimate their computational requirements, identify inefficient algorithms, and recommend suitable improvements.

1.3 Objectives

The objectives of this report are:

- To identify the input size and basic operation of each module.
- To determine the exact or estimated number of operations.
- To analyse algorithmic growth using asymptotic notation.

- To solve recurrence relations using backward substitution, the Master Theorem, and recurrence trees.
- To compare linear search and binary search.
- To prove that a given function belongs to the class $\Theta(n^2)$.
- To estimate execution times for an input size of one million.
- To identify algorithms that exceed the available processing window.
- To recommend improvements for better scalability.

1.4 System Overview

ShopSphere contains five major computational modules.

Module	Function	Main Algorithmic Concern
M1	Duplicate Detector	Comparing product pairs
M2	API Rate Limiter	Auditing previous requests
M3	Product Search	Finding a product in the catalogue
M4	Recommendation Engine	Recursive recommendation generation
M5	Sales Analytics	Recursive processing of sales data

The audit uses the following assumptions:

- Input size: $n = 1{,}000{,}000$, unless otherwise stated.
- Server processing rate: 10^8basic operations per second.
- Available nightly processing window: 600 seconds.
- The algorithms are evaluated mathematically rather than through hardware benchmarking.
- One basic operation is treated as one unit of computational work for estimation purposes.

Important: The estimated execution times are simplified theoretical estimates. Actual execution time depends on hardware, programming language, memory access, data structures, and implementation details.

---

2. SECTION (a): INPUT SIZE, BASIC OPERATION, AND GROWTH RATE

2.1 Meaning of Input Size

Input size represents the amount of data processed by an algorithm.

For ShopSphere, $n$may represent:

- The number of products in the catalogue.
- The number of API requests.
- The number of sales records.
- The number of elements in a search operation.

The exact meaning depends on the module being analysed.

2.2 Meaning of Basic Operation

A basic operation is the fundamental operation that contributes significantly to the running time of an algorithm.

Examples include:

- Comparing two product identifiers.
- Auditing an API request.
- Comparing a search key with a product identifier.
- Combining results from recursive calls.
- Processing a sales-data element.

Counting basic operations helps estimate how an algorithm behaves as the input size increases.

2.3 Analysis of Each Module

M1: Duplicate Detector

Purpose: Identify duplicate products in the catalogue.

The basic implementation compares every product with every other product using nested loops.

```
FOR i ← 0 TO n−1:
    FOR j ← i+1 TO n−1:
        Compare(Product[i], Product[j])
```

The inner loop performs $n - 1$comparisons for the first product, $n - 2$for the second product, and continues until one comparison remains.

Therefore,

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

Using the arithmetic series formula,

$$\boxed{C(n) = \frac{n(n-1)}{2}}$$

Expanding,

$$C(n) = \frac{n^2 - n}{2}$$

The dominant term is $n^2/2$.

Therefore,

$$\boxed{T(n) = \Theta(n^2)}$$

Explanation: The number of comparisons grows approximately as the square of the number of products. Doubling the number of products increases the leading comparison count by approximately four times.

M2: API Rate Limiter

Purpose: Audit previous API requests at exponentially increasing intervals.

The given pseudocode is:

```
i ← 1

WHILE i < n:
    FOR j ← 0 TO i−1:
        Audit(Request[j])

    i ← i × 2
```

The values of $i$are:

1,2,4,8,16, ...

The number of audits in each iteration is equal to the current value of i.

Therefore,

$$A(n) = 1 + 2 + 4 + 8 + \cdots + 2^{k-1}$$

where

$$k = \lceil \log_2 n \rceil$$

For $n > 1$, the geometric series gives:

$$A(n) = 2^k - 1$$

Hence,

$$\boxed{A(n) = 2^{\lceil \log_2 n \rceil} - 1}$$

Since

$$n \leq 2^{\lceil \log_2 n \rceil} < 2n$$

we obtain

$$n - 1 \leq A(n) < 2n - 1$$

Therefore,

$$\boxed{T(n) = \Theta(n)}$$

Explanation: Although the outer loop doubles its variable each time, the inner loop performs a geometrically increasing number of operations. The total is a geometric series, resulting in linear—not logarithmic—time complexity.

M3: Product Search

Purpose: Find a product using its identifier.

Two searching algorithms are considered:

1. Sequential or linear search.

2. Binary search on a sorted index.

For linear search, the algorithm checks elements one by one.

Its worst-case comparison count is:

$$C(n) = n$$

Therefore,

$$\boxed{T(n) = \Theta(n)}$$

For binary search, the search space is halved after each comparison.

Its recurrence relation is:

$$T(n) = T(n/2) + 1$$

The number of times the input can be halved is proportional to $\log_2 n$.

Therefore,

$$\boxed{T(n) = \Theta(\log n)}$$

Explanation: Binary search is more efficient for large datasets, provided the searched data is sorted or an appropriate sorted index is available.

M4: Recommendation Engine

Two versions of the recommendation algorithm are examined.

Current version:

$$T(n) = 4T(n/2) + n$$

Redesigned version:

$$T(n) = 2T(n/2) + n$$

The current algorithm generates four recursive subproblems of half the input size. The redesigned algorithm generates only two.

Using the Master Theorem, the current version has complexity:

$$\boxed{T(n) = \Theta(n^2)}$$

The redesigned version has complexity:

$$\boxed{T(n) = \Theta(n \log n)}$$

Explanation: Reducing the number of recursive subproblems significantly decreases the amount of work required for large inputs.

M5: Sales Analytics

Purpose: Process sales records recursively and combine the results.

The recurrence relation is:

$$T(n) = 2T(n/2) + n\log_2 n$$

At each recursive level, the combined non-recursive work is proportional to $n\log_2 n$, with the logarithmic factor decreasing as the recursion progresses.

The resulting complexity is:

$$\boxed{T(n) = \Theta(n(\log n)^2)}$$

Explanation: This algorithm grows faster than $n \log n$, but slower than n^2. It may still require substantial processing time for very large datasets.

2.4 Summary Table

Module	**Input Size**	**Basic Operation**	**Growth Rate**
M1	Number of products n	Comparing product pairs	$\Theta(n^2)$
M2	Number of requests n	Auditing a request	$\Theta(n)$
M3 Linear Search	Number of products n	Comparing a search key	$\Theta(n)$
M3 Binary Search	Number of sorted entries n	Comparing with the midpoint	$\Theta(\log n)$

Module	Input Size	Basic Operation	Growth Rate
M4 Current	Problem size n	Recursive calls and combining work	$\Theta(n^2)$
M4 Redesigned	Problem size n	Recursive calls and combining work	$\Theta(n \log n)$
M5	Number of sales records n	Recursive processing and combining	$\Theta(n(\log n)^2)$

---

3. SECTION (b): EXACT OPERATION COUNTS

3.1 M1: Duplicate Detector

The duplicate detector compares each product with every product appearing after it.

The exact comparison count is:

$$C(n) = \frac{n(n-1)}{2}$$

For $n = 1{,}000{,}000$,

$$C(n) = \frac{1{,}000{,}000(1{,}000{,}000 - 1)}{2}$$
$$C(n) = \frac{1{,}000{,}000 \times 999{,}999}{2}$$
$$\boxed{C(n) = 499{,}999{,}500{,}000}$$

Therefore, the algorithm performs approximately 500 billion product comparisons.

Execution-Time Estimate

The assumed processing rate is:

$$R = 10^8 \text{ operations/second}$$

The estimated time is:

$$t = \frac{\text{Number of operations}}{\text{Operations per second}}$$

Substituting,

$$t = \frac{499{,}999{,}500{,}000}{100{,}000{,}000}$$

$$\boxed{t = 4{,}999.995 \text{ seconds}}$$

Converting into minutes,

$$t = \frac{4{,}999.995}{60}$$

$$\boxed{t \approx 83.33 \text{ minutes}}$$

The nightly processing window is 600 seconds, equivalent to 10 minutes.

Since

$$4{,}999.995 > 600$$

Conclusion: The duplicate detector exceeds the available processing window and is unsuitable for processing one million products using the current nested-loop approach.

A hash-based approach can often reduce expected duplicate-detection time to $\Theta(n)$, assuming efficient hashing and sufficient memory. Sorting by product identifier is another option, with a typical time complexity of $\Theta(n \log n)$.

3.2 M2: API Rate Limiter

The given algorithm is:

```
i ← 1

WHILE i < n:
  FOR j ← 0 TO i−1:
    Audit(Request[j])

  i ← i × 2
```

The value of $i$doubles in every outer-loop iteration.

The total audit count is:

$$A(n) = 1 + 2 + 4 + \cdots + 2^{k-1}$$

where

$$k = \lceil \log_2 n \rceil$$

Using the geometric series formula,

$$\boxed{A(n) = 2^{\lceil \log_2 n \rceil} - 1}$$

for $n > 1$.

Verification for $n = 1024$

Since

$$1024 = 2^{10}$$

the values of $i$are:

$$1,2,4,8,16,32,64,128,256,512$$

The corresponding number of audits is:

$$A(1024) = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512$$

Using the geometric series formula,

$$A(1024) = 2^{10} - 1$$
$$\boxed{A(1024) = 1023}$$

Therefore, the algorithm performs exactly **1,023 audits** for $n = 1024$.

Is the $\Theta(n \log n)$Claim Correct?

No. The claim that the algorithm has complexity $\Theta(n \log n)$is incorrect.

The audit count is:

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

For $n > 1$,

$$n - 1 \leq A(n) < 2n - 1$$

Thus, the total number of audits is bounded above and below by constant multiples of n.

Therefore,

$$\boxed{A(n) = \Theta(n)}$$

The algorithm has **linear time complexity**, not $\Theta(n \log n)$.

Audit Count for One Million Requests

For

$$n = 1{,}000{,}000$$

we have:

$$2^{19} = 524{,}288$$

and

$$2^{20} = 1{,}048{,}576$$

Therefore,

$$\lceil \log_2(1,000,000) \rceil = 20$$

The exact audit count is:

$$A(n) = 2^{20} - 1$$
$$\boxed{A(n) = 1{,}048{,}575}$$

The estimated execution time is:

$$t = \frac{1{,}048{,}575}{10^8}$$
$$\boxed{t \approx 0.01049 \text{ seconds}}$$

This is approximately 10.49 milliseconds under the simplified operation-rate assumption.

Conclusion: M2 is computationally efficient in terms of total audit operations. However, its actual usefulness as a rate limiter depends on whether this auditing schedule meets the platform's real request-monitoring requirements.

---

4. SECTION (c): SOLVING THE BINARY SEARCH RECURRENCE

4.1 Recurrence Relation

Binary search compares the target with the middle element and continues searching in only one half of the remaining data.

Its recurrence is:

$$T(n) = T(n/2) + 1$$

with base case:

$$T(1) = 1$$

Here:

- $T(n)$represents the number of comparisons for an input of size n.
- $T(n/2)$represents the remaining search work after halving the input.
- The additional 1represents the midpoint comparison.

Assume that $n$is a power of two for the derivation.

4.2 Solution Using Backward Substitution

Starting with:

$$T(n) = T(n/2) + 1$$

Substitute the recurrence into itself:

$$T(n) = [T(n/4) + 1] + 1$$

Therefore,

$$T(n) = T(n/4) + 2$$

Substitute again:

$$T(n) = T(n/8) + 3$$

After $k$substitutions,

$$T(n) = T(n/2^k) + k$$

The recursion reaches the base case when:

$$\frac{n}{2^k} = 1$$

Therefore,

$$n = 2^k$$

Taking logarithms,

$$k = \log_2 n$$

Substituting into the recurrence solution,

$$T(n) = T(1) + \log_2 n$$

Since $T(1) = 1$,

$$\boxed{T(n) = \log_2 n + 1}$$

Therefore, the time complexity is:

$$\boxed{T(n) = \Theta(\log n)}$$

The exact expression assumes the stated recurrence and a power-of-two input size. For other input sizes, the number of binary-search levels is still logarithmic.

4.3 Number of Comparisons for One Million Elements

For

$$n = 1{,}000{,}000$$
$$\log_2(1,000,000) \approx 19.93$$

Therefore,

$$T(n) \approx 19.93 + 1$$
$$T(n) \approx 20.93$$

The worst-case number of comparisons is approximately 20 to 21, depending on the precise search implementation and counting convention.

Using 20 comparisons as a simplified estimate:

$$t = \frac{20}{10^8}$$
$$\boxed{t = 2 \times 10^{-7} \text{ seconds}}$$

This corresponds to approximately 0.2 microseconds under the assumed model.

Actual latency will be affected by memory access, software overhead, and other system factors.

4.4 Iterative Versus Recursive Binary Search

Binary search can be implemented iteratively or recursively.

Iterative Binary Search

The iterative version uses a loop to update the search boundaries.

Its time complexity is:

$$\boxed{\Theta(\log n)}$$

It uses only a constant number of additional variables, such as low, high, and mid.

Therefore, its auxiliary space complexity is:

$$\boxed{\Theta(1)}$$

Recursive Binary Search

The recursive version calls itself on half of the remaining search space.

Its time complexity is:

$$\boxed{\Theta(\log n)}$$

However, every recursive call requires a stack frame. The maximum recursion depth is proportional to $\log n$.

Therefore, its auxiliary space complexity is:

$$\boxed{\Theta(\log n)}$$

Comparison Table

Feature	**Iterative Binary Search**	**Recursive Binary Search**
Time complexity	$\Theta(\log n)$	$\Theta(\log n)$
Auxiliary space	$\Theta(1)$	$\Theta(\log n)$
Uses a loop	Yes	No
Uses recursive calls	No	Yes
Stack overhead	Constant loop state	Recursive call stack
Suitability	Memory-efficient implementation	Clear recursive explanation

Conclusion: Both implementations provide logarithmic search time. The iterative version is generally preferable when minimizing auxiliary memory and recursive-call overhead is important.

---

5. SECTION (d): MASTER THEOREM AND RECURRENCE-TREE ANALYSIS

5.1 The Master Theorem

The Master Theorem is used to solve many divide-and-conquer recurrence relations of the form:

$$T(n) = aT(n/b) + f(n)$$

where:

- $a$is the number of recursive subproblems.

- $b$is the factor by which the input size is reduced.
- $f(n)$represents the non-recursive work performed at each call.

The critical comparison function is:

$$n^{\log_b a}$$

Comparing $f(n)$with this function helps determine the asymptotic growth rate.

5.2 M4 Current Recommendation Algorithm

The current recommendation algorithm has the recurrence:

$$T(n) = 4T(n/2) + n$$

Comparing this with the Master Theorem form,

$$a = 4,\ b = 2,\ f(n) = n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 4}$$

Since

$$\log_2 4 = 2$$

we obtain:

$$n^{\log_2 4} = n^2$$

The non-recursive work is:

$$f(n) = n$$

Since $n$grows polynomially slower than n^2, the first case of the Master Theorem applies.

Therefore,

$$\boxed{T(n) = \Theta(n^2)}$$

Effect of Doubling the Input

For a quadratic algorithm,

$$T(n) \propto n^2$$

If the input changes from $n$to $2n$,

$$\frac{T(2n)}{T(n)} \approx \frac{(2n)^2}{n^2}$$

$$\boxed{\frac{T(2n)}{T(n)} \approx 4}$$

Thus, doubling the input approximately quadruples the computational work.

Exact Recurrence Solution

For $n = 2^k$, with $T(1) = 1$,

$$T(n) = 4T(n/2) + n$$

The solution is:

$$\boxed{T(n) = 2n^2 - n}$$

This expression is exact for power-of-two input sizes under the stated recurrence and base case. For one million elements, it can be used as an approximate estimate.

5.3 M4 Redesigned Recommendation Algorithm

The redesigned algorithm has the recurrence:

$$T(n) = 2T(n/2) + n$$

Therefore,

$$a = 2,\ b = 2,\ f(n) = n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2}$$

Since

$$\log_2 2 = 1$$

we obtain:

$$n^{\log_b a} = n$$

Thus,

$$f(n) = \Theta\left(n^{\log_b a}\right)$$

The non-recursive work and the critical function have the same asymptotic growth.

The second case of the Master Theorem applies.

Therefore,

$$\boxed{T(n) = \Theta(n \log n)}$$

Effect of Doubling the Input

For this algorithm,

$$T(n) \propto n\log_2 n$$

When the input is doubled,

$$\frac{T(2n)}{T(n)} \approx \frac{2n\log_2(2n)}{n\log_2 n}$$

Using

$$\log_2(2n) = \log_2 n + 1$$

we get:

$$\frac{T(2n)}{T(n)} \approx 2\frac{\log_2 n + 1}{\log_2 n}$$

For large n, this ratio approaches 2.

Therefore, doubling the input approximately doubles the work, with a small additional logarithmic increase.

Exact Recurrence Solution

For $n = 2^k$, with $T(1) = 1$,

$$T(n) = 2T(n/2) + n$$

The exact solution is:

$$\boxed{T(n) = n(\log_2 n + 1)}$$

This is valid for power-of-two input sizes under the stated recurrence and base case.

5.4 Comparison of the Two Recommendation Algorithms

Feature	**Current Algorithm**	**Redesigned Algorithm**
Recurrence	$T(n) = 4T(n/2) + n$	$T(n) = 2T(n/2) + n$
Number of recursive subproblems	4	2
Time complexity	$\Theta(n^2)$	$\Theta(n \log n)$
Effect of doubling input	Approximately 4 times the work	Slightly more than 2 times the work
Scalability	Poor for large inputs	Significantly better
Recommendation	Redesign required	Preferable, subject to correctness

The redesigned algorithm is asymptotically more efficient. However, a practical implementation must also preserve the required recommendation functionality and output quality.

5.5 M5: Sales Analytics Using the Extended Master Theorem

The recurrence is:

$$T(n) = 2T(n/2) + n\log_2 n$$

Comparing it with:

$$T(n) = aT(n/b) + f(n)$$

we obtain:

$$a = 2,\ b = 2,\ f(n) = n\log_2 n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Therefore,

$$f(n) = n\log_2 n$$

or equivalently,

$$f(n) = \Theta(n \log n)$$

The non-recursive work is larger than the critical function by a logarithmic factor.

Using the extended Master Theorem, the applicable case gives:

$$T(n) = \Theta(n\log^{1+1} n)$$

Therefore,

$$\boxed{T(n) = \Theta(n(\log n)^2)}$$

Verification Using a Recurrence Tree

Assume $n = 2^k$.

At level 0, the work is:

$$n\log_2 n = nk$$

At level 1, there are two subproblems of size $n/2$.

The total work is:

$$2\left(\frac{n}{2}\log_2 \frac{n}{2}\right)$$

Since

$$\log_2 \frac{n}{2} = k - 1$$

the total is:

$$n(k-1)$$

At level 2, there are four subproblems of size $n/4$.

The total work is:

$$4\left(\frac{n}{4}\log_2 \frac{n}{4}\right)$$

Therefore, the work is:

$$n(k-2)$$

The pattern continues.

Level	**Number of Subproblems**	**Size of Each Subproblem**	**Total Work**
0	1	n	nk
1	2	$n/2$	$n(k-1)$
2	4	$n/4$	$n(k-2)$
3	8	$n/8$	$n(k-3)$
$k-1$	2^{k-1}	2	n
k	2^k	1	Leaf cost

The internal-level work is:

$$n[k + (k-1) + (k-2) + \cdots + 1]$$

Using the sum of the first $k$positive integers,

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}$$

we obtain:

$$W_{\text{internal}} = n\frac{k(k+1)}{2}$$

If every leaf has base-case cost $T(1) = 1$, there are $n$leaves and their total cost is n.

Therefore,

$$T(n) = n\frac{k(k+1)}{2} + n$$

Since $k = \log_2 n$,

$$\boxed{T(n) = \frac{n}{2}\log_2 n(\log_2 n + 1) + n}$$

This expression is exact for powers of two under the stated recurrence and base case.

Its dominant term is:

$$\frac{n}{2}(\log_2 n)^2$$

Therefore,

$$\boxed{T(n) = \Theta(n(\log n)^2)}$$

Conclusion for M5

The recurrence-tree analysis confirms the result obtained using the extended Master Theorem. Sales analytics requires more work than a typical $\Theta(n \log n)$algorithm, so its execution cost must be considered when planning large batch-processing jobs.

---

6. SECTION (e): LINEAR SEARCH VERSUS BINARY SEARCH

6.1 Linear Search

Linear search examines elements one by one until the target is found or all elements have been checked.

Best Case

The target is the first element.

Number of comparisons:

$$\boxed{C_{\text{best}}(n) = 1}$$

Therefore,

$$\boxed{T_{\text{best}}(n) = \Theta(1)}$$

Worst Case

The target is the final element or is absent from the dataset.

Number of comparisons:

$$\boxed{C_{\text{worst}}(n) = n}$$

Therefore,

$$\boxed{T_{\text{worst}}(n) = \Theta(n)}$$

Average Case for a Successful Search

Assume that the target is equally likely to appear at any of the $n$positions.

If the target is at position 1, one comparison is needed. If it is at position 2, two comparisons are needed. This continues until position n.

The average comparison count is:

$$C_{\text{avg}}(n) = \frac{1 + 2 + \cdots + n}{n}$$

Using the arithmetic series formula,

$$C_{\text{avg}}(n) = \frac{n(n+1)}{2n}$$

Therefore,

$$\boxed{C_{\text{avg}}(n) = \frac{n+1}{2}}$$

The average successful-search time complexity is:

$$\boxed{T_{\text{avg}}(n) = \Theta(n)}$$

6.2 Binary Search

Binary search requires sorted data or a suitable sorted index.

It compares the target with the middle element and continues in the half that may contain the target.

Best Case

The target is the middle element of the initial search range.

Number of comparisons:

$$\boxed{C_{\text{best}}(n) = 1}$$

Therefore,

$$\boxed{T_{\text{best}}(n) = \Theta(1)}$$

Worst Case

The algorithm continues halving the search space until the target is found or the search range becomes empty.

The number of comparisons grows logarithmically.

$$\boxed{C_{\text{worst}}(n) = \Theta(\log n)}$$

Therefore,

$$\boxed{T_{\text{worst}}(n) = \Theta(\log n)}$$

Average Case

For a standard balanced binary search, the number of comparisons is proportional to the depth of the search tree.

Therefore, the average successful-search complexity is:

$$\boxed{T_{\text{avg}}(n) = \Theta(\log n)}$$

The precise average count depends on the target distribution and the implementation's comparison convention.

6.3 Search Comparison Table

Case	Linear Search	Binary Search
Best-case comparisons	1	1
Average successful search	$(n+1)/2$	$\Theta(\log n)$
Worst-case comparisons	n	$\Theta(\log n)$
Worst-case time complexity	$\Theta(n)$	$\Theta(\log n)$
Requires sorted data	No	Yes
Suitable for large sorted catalogues	Less efficient	More efficient

Conclusion: Binary search is generally more efficient for repeated product lookups in a large, sorted catalogue. Linear search remains useful for small datasets or situations where maintaining a sorted index is unnecessary.

6.4 Expected Linear Search Comparisons When the Target May Be Absent

Let:

- n= total number of elements.
- p= probability that the target exists.
- $1 - p$= probability that the target is absent.

Assume that, when the target exists, it is equally likely to appear at any position.

The average number of comparisons when the target exists is:

$$C_{\text{present}} = \frac{n+1}{2}$$

When the target is absent, every element must be checked:

$$C_{\text{absent}} = n$$

The expected number of comparisons is:

$$E[C] = pC_{\text{present}} + (1-p)C_{\text{absent}}$$

Substituting,

$$E[C] = p\frac{n+1}{2} + (1-p)n$$

Expanding,

$$E[C] = \frac{p(n+1)}{2} + n - pn$$

Simplifying,

$$\boxed{E[C] = n\left(1 - \frac{p}{2}\right) + \frac{p}{2}}$$

Evaluation for $n = 1,000,000$ and $p = 0.5$

Given:

$$n = 1{,}000{,}000$$
$$p = 0.5$$

Substitute into the expected-comparison formula:

$$E[C] = 1{,}000{,}000\left(1 - \frac{0.5}{2}\right) + \frac{0.5}{2}$$
$$E[C] = 1{,}000{,}000(0.75) + 0.25$$

Therefore,

$$\boxed{E[C] = 750{,}000.25}$$

The expected comparison count is approximately **750,000 comparisons per search**.

The expected number is fractional because it represents a probability-weighted average across many searches, not the comparison count of an individual search.

Estimated Time

Using the assumed processing rate,

$$t = \frac{750{,}000.25}{10^8}$$

$$\boxed{t \approx 0.0075 \text{ seconds}}$$

This corresponds to approximately 7.5 milliseconds under the simplified model.

Conclusion: Even when the target has a 50% chance of being absent, linear search may perform hundreds of thousands of comparisons for a catalogue containing one million products. Binary search can reduce the number of comparisons to roughly 20–21, provided the data is suitably indexed.

---

7. SECTION (f): PROVING THAT $f(n)$IS $\Theta(n^2)$

Consider the function:

$$\boxed{f(n) = 3n^2 + 10n\log_2 n + 50}$$

We must prove that:

$$f(n) = \Theta(n^2)$$

We must also show that:

$$f(n) \notin O(n)$$

7.1 Definition of Big-Theta

A function $f(n)$belongs to $\Theta\big(g(n)\big)$if there exist positive constants c_1, c_2, and $n_0$such that:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

for every $n \geq n_0$.

For this problem,

$$g(n) = n^2$$

Therefore, we need to find positive constants satisfying:

$$c_1 n^2 \leq 3n^2 + 10n\log_2 n + 50 \leq c_2 n^2$$

for all sufficiently large n.

7.2 Lower Bound

We have:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

For $n \geq 1$,

$$\log_2 n \geq 0$$

Therefore, the remaining terms are non-negative, giving:

$$f(n) \geq 3n^2$$

Choose:

$$\boxed{c_1 = 3}$$

Then:

$$\boxed{3n^2 \leq f(n)}$$

Thus, the required lower bound is satisfied for $n \geq 1$.

7.3 Upper Bound

For every integer $n \geq 1$, we can use the inequality:

$$\log_2 n \leq n$$

Multiplying both sides by $10n$,

$$10n\log_2 n \leq 10n^2$$

Also, since $n \geq 1$,

$$50 \leq 50n^2$$

Substitute these inequalities into the original function:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

Therefore,

$$f(n) \leq 3n^2 + 10n^2 + 50n^2$$

Combining the terms,

$$f(n) \leq 63n^2$$

Choose:

$$\boxed{c_2 = 63}$$

Thus,

$$\boxed{f(n) \leq 63n^2}$$

for $n \geq 1$.

7.4 Final Big-Theta Proof

We have established:

$$3n^2 \leq f(n) \leq 63n^2$$

for every integer $n \geq 1$.

Therefore, valid constants are:

$$\boxed{c_1 = 3,\ c_2 = 63,\ n_0 = 1}$$

By the formal definition of Big-Theta,

$$\boxed{f(n) = \Theta(n^2)}$$

Hence, the function grows asymptotically at a quadratic rate.

7.5 Proving That $f(n) \notin O(n)$

Suppose, for contradiction, that:

$$f(n) \in O(n)$$

By the definition of Big-O, there must exist constants $c > 0$and $n_0$such that:

$$f(n) \leq cn$$

for every $n \geq n_0$.

However, from the original function,

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

we know that:

$$f(n) \geq 3n^2$$

Combining the two inequalities gives:

$$3n^2 \leq cn$$

For $n > 0$, divide both sides by n:

$$3n \leq c$$

Therefore,

$$n \leq \frac{c}{3}$$

But $c$is a fixed constant, while $n$can grow arbitrarily large. Consequently, this inequality cannot hold for every sufficiently large n.

This contradicts the assumption that $f(n) \in O(n)$.

Therefore,

$$\boxed{f(n) \notin O(n)}$$

Final Conclusion

The function

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

has a quadratic dominant term. The lower- and upper-bound proofs establish that:

$$\boxed{f(n) = \Theta(n^2)}$$

The function cannot belong to $O(n)$, because its quadratic growth eventually exceeds every constant multiple of a linear function.

---

8. SECTION (g): SCALABILITY AUDIT FOR ONE MILLION ELEMENTS

8.1 Given Parameters

The audit uses the following parameters:

Parameter	Value
Input size	1,000,000
Server processing rate	10^8 operations/second
Nightly processing window	600 seconds
Processing window in minutes	10 minutes

The estimated execution time is calculated using:

$$\boxed{\text{Estimated Time} = \frac{\text{Estimated Operations}}{10^8}}$$

The following estimates treat each counted operation as one unit of work. They do not represent measured production latency.

8.2 M1: Duplicate Detector

The exact comparison count is:

$$C(n) = \frac{n(n-1)}{2}$$

For $n = 1{,}000{,}000$,

$$C(n) = 499{,}999{,}500{,}000$$

Estimated time:

$$t = \frac{499{,}999{,}500{,}000}{100{,}000{,}000}$$

$$\boxed{t \approx 4{,}999.995 \text{ seconds}}$$

Converting to minutes:

$$\boxed{t \approx 83.33 \text{ minutes}}$$

Since the allowed time is 10 minutes, M1 fails the scalability requirement.

Recommended Redesign

Use a hash table to group or identify products by a unique identifier.

Under the usual assumptions of efficient hashing, duplicate detection can often be performed in expected:

$$\boxed{\Theta(n)}$$

time, with additional memory usage.

Alternatively, sorting the product identifiers and checking adjacent entries can achieve:

$$\boxed{\Theta(n \log n)}$$

time with a suitable sorting algorithm.

8.3 M2: API Rate Limiter

The audit count is:

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

For one million requests,

$$A(n) = 2^{20} - 1$$
$$\boxed{A(n) = 1{,}048{,}575}$$

Estimated time:

$$t = \frac{1{,}048{,}575}{100{,}000{,}000}$$
$$\boxed{t \approx 0.01049 \text{ seconds}}$$

This is approximately 10.49 milliseconds under the simplified model.

Therefore, M2 satisfies the computational time limit in this analysis.

However, this conclusion applies only to the provided auditing pseudocode. A real API rate limiter must also enforce the required policy, maintain correct request counts, and handle concurrent requests safely.

8.4 M3: Binary Search

Binary search has logarithmic time complexity:

$$T(n) = \Theta(\log n)$$

For one million elements, approximately 20–21 comparisons are sufficient in the worst case for a conventional implementation.

Using 20 comparisons as a simplified estimate:

$$t = \frac{20}{100{,}000{,}000}$$
$$\boxed{t = 0.0000002 \text{ seconds}}$$

The theoretical estimate is well below 600 seconds.

However, the estimate covers the search comparisons only. It does not include the time required to build, sort, or update the product index.

8.5 M4: Current Recommendation Algorithm

The recurrence is:

$$T(n) = 4T(n/2) + n$$

Its asymptotic complexity is:

$$\Theta(n^2)$$

For powers of two, with $T(1) = 1$, the exact solution is:

$$T(n) = 2n^2 - n$$

Using the same expression as an approximation for one million elements,

$$T(n) \approx 2(1,000,000)^2 - 1{,}000{,}000$$
$$\boxed{T(n) \approx 1{,}999{,}999{,}000{,}000}$$

Estimated time:

$$t \approx \frac{1{,}999{,}999{,}000{,}000}{100{,}000{,}000}$$
$$\boxed{t \approx 19{,}999.99 \text{ seconds}}$$

Converting into hours,

$$t \approx \frac{19{,}999.99}{3{,}600}$$
$$\boxed{t \approx 5.56 \text{ hours}}$$

This is much greater than the available 10-minute window.

Therefore, the current recommendation algorithm fails the scalability requirement.

Note: One million is not a power of two, so this recurrence expression is used as a model-based approximation for that input size.

8.6 M4: Redesigned Recommendation Algorithm

The redesigned recurrence is:

$$T(n) = 2T(n/2) + n$$

Its complexity is:

$$\Theta(n \log n)$$

For powers of two, with $T(1) = 1$, the exact solution is:

$$T(n) = n(\log_2 n + 1)$$

For one million elements,

$$\log_2(1,000,000) \approx 19.9316$$

Therefore,

$$T(n) \approx 1{,}000{,}000(19.9316 + 1)$$
$$\boxed{T(n) \approx 20{,}931{,}569}$$

Estimated time:

$$t \approx \frac{20{,}931{,}569}{100{,}000{,}000}$$
$$\boxed{t \approx 0.2093 \text{ seconds}}$$

This estimate is well below the 600-second limit.

The redesigned recurrence demonstrates a substantial theoretical improvement over the current quadratic version.

However, the algorithm should be validated to ensure that the reduced number of recursive subproblems preserves the recommendation system's required functionality.

8.7 M5: Sales Analytics

The recurrence is:

$$T(n) = 2T(n/2) + n\log_2 n$$

Its complexity is:

$$\Theta(n(\log n)^2)$$

For powers of two, with $T(1) = 1$, the recurrence-tree solution is:

$$T(n) = \frac{n}{2}\log_2 n(\log_2 n + 1) + n$$

Using the continuous logarithmic expression as an approximation for one million elements,

$$\log_2(1,000,000) \approx 19.9316$$

Therefore,

$$T(n) \approx \frac{1{,}000{,}000}{2}(19.9316)(20.9316) + 1{,}000{,}000$$

This gives approximately:

$$\boxed{T(n) \approx 209{,}500{,}000 \text{ operations}}$$

Estimated time:

$$t \approx \frac{209{,}500{,}000}{100{,}000{,}000}$$

$$\boxed{t \approx 2.10 \text{ seconds}}$$

This is a recurrence-based approximation, not an exact operation count for one million elements.

Under the simplified model, M5 fits within the 600-second processing window.

However, actual sales analytics may include data loading, aggregation, database access, and report generation, which are not represented by this recurrence alone.

8.8 Consolidated Scalability Audit

Module	**Estimated Operations at $n =$ 1,000,000**	**Estimated Time**	**Within 600 Seconds?**
M1 Duplicate Detector	499,999,500,000	4,999.995 seconds	No
M2 API Rate Limiter	1,048,575	0.01049 seconds	Yes
M3 Binary Search	Approximately 20 comparisons for the simplified estimate	0.0000002 seconds	Yes

Module	Estimated Operations at $n = 1{,}000{,}000$	Estimated Time	Within 600 Seconds?
M4 Current Recommendation	Approximately 1,999,999,000,000	19,999.99 seconds	No
M4 Redesigned Recommendation	Approximately 20,931,569	0.2093 seconds	Yes
M5 Sales Analytics	Approximately 209,500,000	2.10 seconds	Yes

Interpretation: M1 and the current M4 recommendation algorithm exceed the nightly processing window under the assumed operation-rate model. The other modules fit within the theoretical time limit.

These estimates should be interpreted cautiously because an operation-rate assumption does not capture all hardware and implementation costs.

8.9 Scalability Recommendations

Recommendation 1: Improve Duplicate Detection

Replace nested-loop comparisons with a hash-based method when suitable identifiers are available.

Expected benefit: Reduces expected processing time from quadratic to linear.

Recommendation 2: Maintain a Sorted Product Index

Use binary search for repeated product lookups where the index can be maintained efficiently.

Expected benefit: Reduces search time from linear to logarithmic.

Recommendation 3: Redesign the Recommendation Engine

Reduce unnecessary recursive subproblems and investigate suitable divide-and-conquer or indexing strategies.

Expected benefit: Improves asymptotic complexity from $\Theta(n^2)$to $\Theta(n \log n)$under the redesigned recurrence.

Recommendation 4: Validate API Rate-Limiting Requirements

Ensure that the auditing schedule matches the platform's actual rate-limiting policy and concurrency requirements.

Expected benefit: Maintains efficient processing without compromising correctness.

Recommendation 5: Optimise Sales Analytics

Investigate opportunities to reuse intermediate results, reduce repeated computations, and process independent batches where appropriate.

Expected benefit: Helps maintain predictable execution time as the transaction dataset grows.

Recommendation 6: Perform Real Benchmarks

Measure execution time using realistic datasets and the actual implementation.

Record:

- Runtime.
- Memory usage.
- Input size.
- Number of operations, where measurable.
- Data-loading overhead.
- Index construction and update costs.

Expected benefit: Confirms whether the theoretical estimates accurately reflect production performance.

---

9. OVERALL FINDINGS

The scalability audit demonstrates that algorithm selection has a major influence on system performance.

The main findings are:

1. The nested-loop duplicate detector performs approximately 500 billion comparisons for one million products and exceeds the nightly processing window.
2. The API rate-limiter pseudocode performs 1,023 audits for $n = 1024$, demonstrating that its total audit count is $\Theta(n)$, not $\Theta(n \log n)$.
3. Binary search requires only logarithmically many comparisons when the searched data is sorted.
4. The current recommendation recurrence has quadratic complexity, while the redesigned recurrence has $n \log n$complexity.
5. The sales analytics recurrence has complexity $\Theta(n(\log n)^2)$, as confirmed by both the extended Master Theorem and recurrence-tree analysis.
6. The function $3n^2 + 10n\log_2 n + 50$belongs to $\Theta(n^2)$and does not belong to $O(n)$.

7. The theoretical time estimates identify M1 and the current M4 algorithm as the main scalability bottlenecks.

These findings demonstrate the importance of combining mathematical analysis with practical validation when designing scalable software systems.

---

10. CONCLUSION

This report presented a detailed scalability audit of the ShopSphere e-commerce platform using the principles of Design and Analysis of Algorithms.

The analysis examined duplicate detection, API request auditing, product searching, recommendation generation, and sales analytics. Each module was evaluated in terms of input size, basic operations, time complexity, and estimated execution time.

The duplicate detector was identified as inefficient because its quadratic comparison count becomes extremely large for one million products. The current recommendation algorithm also demonstrated poor scalability due to its quadratic recurrence. In contrast, binary search and the redesigned recommendation recurrence exhibited substantially better asymptotic growth.

The API rate-limiting analysis established that the given pseudocode performs a geometrically increasing number of audits whose total is linear in the input size. The binary-search recurrence was solved using backward substitution, and the sales analytics recurrence was analysed using both the extended Master Theorem and a recurrence tree.

The formal asymptotic proof established that the given function belongs to $\Theta(n^2)$and cannot be bounded above by a constant multiple of n.

Finally, the scalability audit showed that algorithmic improvements can significantly reduce computational work. Hash-based duplicate detection, sorted indexing, a redesigned recommendation algorithm, and systematic performance testing are suitable strategies for improving the platform.

In conclusion, selecting an appropriate algorithm is essential for building efficient, reliable, and scalable e-commerce applications.

REFERENCES

You may include the following general references at the end of your report.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. MIT Press.
2. Kleinberg, J., and Tardos, É. *Algorithm Design*. Pearson.

3. Course notes and lecture materials for Design and Analysis of Algorithms.
4. Assessment instructions and problem statements supplied for the ShopSphere scalability audit